

Dynamic Queue in Java

Mostafa Faik

October 6, 2024

Introduction

Queues are a fundamental data structure that operate on a **First-In-First-Out (FIFO)** basis. This structure operation removes first the element that was the first one added. In this assignment, we explore how to implement a **queue** using an array, which is more efficient in terms of memory use than a **linked list**. However, the implementation can be more complex due to edge cases such as array resizing and handling **wraparound** when the array reaches its capacity.

We aim to efficiently manage memory using an array with **wraparound** to prevent the costly operation of shifting elements. We will also cover how to dynamically resize the array when it becomes full, ensuring the **queue** can grow as needed.

Basic Array-Based Queue Implementation

Concept of Queue Using Array

A **queue** can be implemented using an array where:

- **Enqueue** adds an element at the back of the **queue**, i.e, add an element at position k and increment k .
- **Dequeue** removes an element from the front, where we remove an element at position i and increment i .
- Two pointers, i (front) and k (end), keep track of where elements should be added and removed. This is similar to the head and tail in the **linked list** implementation.

However, when the array is full or nearly full, we run into problems. The array might need to be resized, or we may want to reuse free space from the start of the array when elements are **dequeued**.

Initially, a naive approach might involve shifting all elements forward after each **dequeue**. However, this is inefficient and costly, as it has $O(n)$ time complexity for each **dequeue** operation. We want to avoid this overhead.

Using an Array with Wraparound

The solution to this inefficiency is to implement a **wraparound** mechanism. **Wraparound** allows us to reuse free slots in the array when the end of the array is reached. Instead of shifting elements forward, we treat the array as circular. When the k pointer reaches the end of an array, it will then "wrap around" to the start of that array only if there is free space.

Wraparound Implementation Steps

Tracking the Front and End (We maintain two indices):

- i : Points to the front of the queue.
- k : Points to the next free slot where an element will be **enqueued**.

Both i and k are updated using the modulo operation ($\text{mod } n$), where n is the size of the array. This allows the **queue** to wrap around when either pointer reaches the end of the array.

Wraparound Example:

Suppose that the array that we are working with, has size $n = 5$, and after doing some operations, the i pointer is at position 3 and the k pointer is at position 4. After an **enqueue** operation, k is incremented and wraps around to 0, indicating that the first slot in the array is now free for **enqueueing**.

Edge Cases and Challenges

Full Queue Detection:

If $i == k$ after an **enqueue** operation and the array is not empty, this indicates that the array is full. At this point, we need to resize the array.

Empty Queue Detection:

If $i == k$ and no elements are in the **queue**, this means the **queue** is empty, and no further **dequeue** operations should be allowed.

Dynamic Resizing of the Array

In addition to the `wraparound` logic, we must handle cases where the array becomes full. To ensure that our `queue` can grow dynamically, we implement a resizing strategy. When the array is full:

- Allocate a new array with a larger size (typically double the original size).
- Copy all the existing elements from the old array to the new array, taking into account the `wraparound` logic.
- Update the i and k pointers to reflect their positions in the new array.

Resizing Process

- **Allocating a New Array:** A new array is created with double the size of the original array.
- **Copying Elements:** We copy elements from the old array to the new array, starting from i and wrapping around if needed, until we reach k .
- **Updating Indices:** After copying, the i pointer is set to 0 , and k is set to the number of elements that were in the old array.

Nullifying Dequeued Elements

To ensure efficient memory use, we must set `dequeued` positions to `null`. This prevents memory leaks and ensures that old references are not inadvertently retained, allowing the Java `garbage collector` to clean up unused memory.

If we do not set `dequeued` positions to `null`, those elements may still be held in memory, even though they are no longer part of the `queue`. By **nullifying** these positions, we free up memory and reduce the likelihood of performance issues due to excessive memory consumption.

Dynamic Queue Implementation

Initialization:

- Create an array of size n to store the `queue` elements.
- Initialize both i (front) and k (back) to 0 , indicating an empty `queue`.

Enqueue Operation:

- Add the new element at position k .
- Increment k using $k = (k + 1) \% n$ to handle **wraparound**.
- If the **queue** is full ($i == k$), resize the array before adding the new element.

Dequeue Operation:

- Return the element at position i and set that position to *null*.
- Increment i using $i = (i + 1) \% n$ to handle **wraparound**.
- If $i == k$, the **queue** is empty.

Resizing:

- Allocate a new array with double the size.
- Copy elements from the old array, adjusting for **wraparound**.
- Update i to 0 and k to the number of elements in the **queue**.

Dynamic Queue Benchmark

To assess the performance of the **dynamic array-based queue**, we conducted benchmarks with different **queue** sizes, measuring the time taken for both **enqueue** and **dequeue** operations. The following table shows the result of the benchmark:

Case	size	Enqueue Time	Dequeue Time
1	100	09.50 ns	07.41 ns
2	200	22.81 ns	11.24 ns
3	400	86.07 ns	23.51 ns
4	800	18.36 ns	11.36 ns
5	1600	38.43 ns	29.27 ns

Table 1: Array Queue Benchmark Result

We can observe that, the **enqueue** times show some variability, which might be due to factors like memory allocation. We can see that there is a spike in case *400* for the **enqueue** time (*86.07 ns*), this indicates that the array had a resizing event, where the **queue** needed to allocate more memory and copy existing elements. For cases *800* and *1600*, we have lower **enqueue**

times (18.36 ns and 38.43 ns), this indicates that after resizing, the `enqueue` operation returned to its amortized constant time of $O(1)$.

The `dequeue` times are fairly consistent, with the highest time being 29.27 ns for case size 1600 . `Dequeue` operations follow an expected constant-time pattern of $O(1)$, with the exception of some small fluctuations that may be attributed to system performance factors or garbage collection.

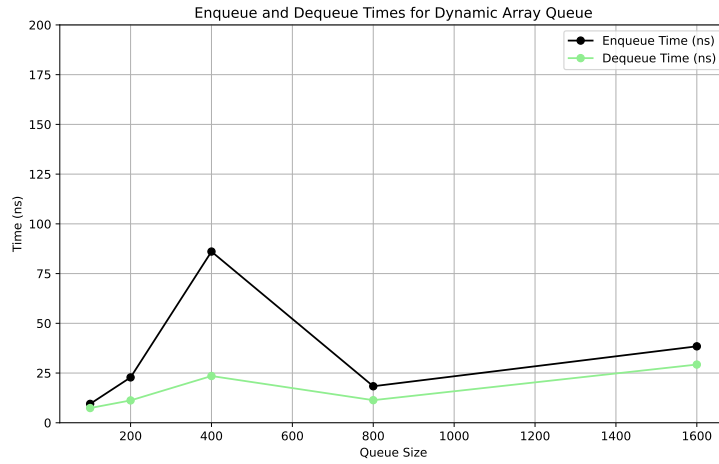


Figure 1: Array Queue Graph Result

Aspect	Linked List Queue	Array-Based Queue
Enqueue Time Complexity	$O(1)$ with tail pointer, $O(n)$ without	Amortized $O(1)$, occasional $O(n)$ (resizing)
Dequeue Time Complexity	$O(1)$	$O(1)$ (with wraparound)
Memory Usage	Dynamic, each new element requires a node allocation	Fixed size initially, resizing requires copying elements
Performance Stability	Stable if tail pointer is used, slow without it	Stable with occasional spikes due to resizing
Best for	When constant-time enqueue and dequeue are needed, and memory allocation is less of a concern	When memory efficiency is critical and amortized performance is acceptable

Table 2: Comparison between Linked List Queue and Array-Based Queue

Conclusion

In this assignment, we implemented a `dynamic queue` using an array with a `wraparound mechanism`. This approach optimizes memory usage and minimizes the overhead associated with shifting elements. By dynamically resizing the array when needed, we ensure that the `queue` can grow efficiently while maintaining constant-time `enqueue` and `dequeue` operations. Setting `dequeued` positions to `null` helps with memory management, ensuring that unnecessary references are cleaned up by the garbage collector.

The `dynamic array-based queue` demonstrates an important trade-off between complexity and performance. While the `wraparound` logic adds complexity, it allows us to make efficient use of space and avoid the costly operation of shifting elements.

This assignment highlights the importance of understanding the underlying mechanics of data structures and how to manage memory efficiently in high-performance applications.